

# SQL Functions

Name: Nomit Bagaria

Assignment Name: SQL Functions

Mob.No: 9828255290

Email: [bagarianomit@gmail.com](mailto:bagarianomit@gmail.com)

## SQL Code to Create the Table and insert the Data :

```
CREATE TABLE StudentPerformance (  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    course VARCHAR(30),  
    score INT,  
    attendance INT,  
    mentor VARCHAR(50),  
    join_date DATE,  
    city VARCHAR(50)  
);  
INSERT INTO StudentPerformance  
(student_id, name, course, score, attendance, mentor, join_date, city)  
VALUES  
(101,'Aarav Mehta','Data Science',88,92,'Dr. Sharma','2023-06-12','Mumbai'),  
(102,'Riya Singh','Data Science',76,85,'Dr. Sharma','2023-07-01','Delhi'),  
(103,'Kabir Khan','Python',91,96,'Ms. Nair','2023-06-20','Mumbai'),  
(104,'Tanvi Patel','SQL',84,89,'Mr. Iyer','2023-05-30','Bengaluru'),  
(105,'Ayesha Khan','Python',67,81,'Ms. Nair','2023-07-10','Hyderabad'),  
(106,'Dev Sharma','SQL',73,78,'Mr. Iyer','2023-05-28','Pune'),  
(107,'Arjun Verma','Tableau',95,98,'Ms. Kapoor','2023-06-15','Delhi'),  
(108,'Meera Pillai','Tableau',82,87,'Ms. Kapoor','2023-06-18','Kochi'),  
(109,'Nikhil Rao','Data Science',79,82,'Dr. Sharma','2023-07-05','Chennai'),  
(110,'Priya Desai','SQL',92,94,'Mr. Iyer','2023-05-27','Bengaluru'),  
(111,'Siddharth Jain','Python',85,90,'Ms. Nair','2023-07-02','Mumbai'),  
(112,'Sneha Kulkarni','Tableau',74,83,'Ms. Kapoor','2023-06-10','Pune'),  
(113,'Rohan Gupta','SQL',89,91,'Mr. Iyer','2023-05-25','Delhi'),  
(114,'Ishita Joshi','Data Science',93,97,'Dr. Sharma','2023-06-25','Bengaluru'),  
(115,'Yuvraj Rao','Python',71,84,'Ms. Nair','2023-07-12','Hyderabad');
```

**Q1.Create a ranking of students based on score (highest first).**

**Answer:**

```
SELECT
    student_id,
    name,
    course,
    score,
    RANK() OVER (ORDER BY score DESC) AS student_rank
FROM StudentPerformance;
```

The screenshot shows a SQL query editor with the following code:

```
246     course,
247     score,
248     RANK() OVER (ORDER BY score DESC) AS student_rank
249 FROM StudentPerformance;
250
```

Below the editor is a 'Result Grid' showing the output of the query. The grid has columns: student\_id, name, course, score, and student\_rank. The data is sorted by score in descending order.

| student_id | name           | course       | score | student_rank |
|------------|----------------|--------------|-------|--------------|
| 107        | Arjun Verma    | Tableau      | 95    | 1            |
| 114        | Ishita Joshi   | Data Science | 93    | 2            |
| 110        | Priya Desai    | SQL          | 92    | 3            |
| 103        | Kabir Khan     | Python       | 91    | 4            |
| 113        | Rohan Gupta    | SQL          | 89    | 5            |
| 101        | Aarav Mehta    | Data Science | 88    | 6            |
| 111        | Siddharth Jain | Python       | 85    | 7            |
| 104        | Tanvi Patel    | SQL          | 84    | 8            |
| 108        | Meera Pillai   | Tableau      | 82    | 9            |
| 109        | Nikhil Rao     | Data Science | 79    | 10           |
| 102        | Riya Singh     | Data Science | 76    | 11           |
| 112        | Sneha Kulkarni | Tableau      | 74    | 12           |
| 106        | Dev Sharma     | SQL          | 73    | 13           |
| 115        | Yuvraj Rao     | Python       | 71    | 14           |
| 105        | Ayesha Khan    | Python       | 67    | 15           |

**Q2.Show each student's score and the previous student's score (based on score order).**

**Answer:**

```
SELECT
    student_id,
    name,
    score,
    LAG(score) OVER (ORDER BY score DESC) AS previous_score
FROM StudentPerformance;
```

The screenshot shows a SQL query editor with the following code:

```
251 • SELECT
252     student_id,
253     name,
254     score,
255     LAG(score) OVER (ORDER BY score DESC) AS previous_score
256 FROM StudentPerformance;
257
```

Below the editor is a 'Result Grid' showing the output of the query. The grid has columns: student\_id, name, score, and previous\_score. The data is sorted by score in descending order. The 'previous\_score' column shows the score of the student with the next highest score.

| student_id | name           | score | previous_score |
|------------|----------------|-------|----------------|
| 107        | Arjun Verma    | 95    | 93             |
| 114        | Ishita Joshi   | 93    | 92             |
| 110        | Priya Desai    | 92    | 91             |
| 103        | Kabir Khan     | 91    | 89             |
| 113        | Rohan Gupta    | 89    | 88             |
| 101        | Aarav Mehta    | 88    | 85             |
| 111        | Siddharth Jain | 85    | 84             |
| 104        | Tanvi Patel    | 84    | 82             |
| 108        | Meera Pillai   | 82    | 79             |
| 109        | Nikhil Rao     | 79    | 76             |
| 102        | Riya Singh     | 76    | 74             |
| 112        | Sneha Kulkarni | 74    | 73             |
| 106        | Dev Sharma     | 73    | 71             |
| 115        | Yuvraj Rao     | 71    | 67             |
| 105        | Ayesha Khan    | 67    |                |

**Q3. Convert all student names to uppercase and extract the month name from join\_date.**

**Answer:**

```
SELECT
    UPPER(name) AS student_name,
    MONTHNAME(join_date) AS join_month
FROM StudentPerformance;
```

The screenshot shows a SQL query editor with the following code:

```
258 • SELECT
259     UPPER(name) AS student_name,
260     MONTHNAME(join_date) AS join_month
261 FROM StudentPerformance;
262
263
```

Below the editor is a 'Result Grid' showing the output of the query. The grid has columns: student\_name and join\_month. The student names are in uppercase, and the join\_month is the month name extracted from the join\_date.

| student_name   | join_month |
|----------------|------------|
| AARAV MEHTA    | June       |
| RIYA SINGH     | July       |
| KABIR KHAN     | June       |
| TANVI PATEL    | May        |
| AYESHA KHAN    | July       |
| DEV SHARMA     | May        |
| ARJUN VERMA    | June       |
| MEERA PILLAI   | June       |
| NIKHIL RAO     | July       |
| PRIYA DESAI    | May        |
| SIDDHARTH J... | July       |
| SNEHA KULKA... | June       |
| ROHAN GUPTA    | May        |
| ISHITA JOSHI   | June       |
| YUVRAJ RAO     | July       |

**Q4. Show each student's name and the next student's attendance (ordered by attendance).**

**Answer:**

```
SELECT
    name,
    attendance,
    LEAD(attendance) OVER (ORDER BY attendance)
    AS next_attendance
FROM StudentPerformance;
```

263 • **SELECT**

264     name,

265     attendance,

266     LEAD(attendance) OVER (ORDER BY attendance) AS next\_at

267     FROM StudentPerformance;

268

269

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

|   | name           | attendance | next_attendance |
|---|----------------|------------|-----------------|
| ▶ | Dev Sharma     | 78         | 81              |
|   | Ayesha Khan    | 81         | 82              |
|   | Nikhil Rao     | 82         | 83              |
|   | Sneha Kulkarni | 83         | 84              |
|   | Yuvraj Rao     | 84         | 85              |
|   | Riya Singh     | 85         | 87              |
|   | Meera Pillai   | 87         | 89              |
|   | Tanvi Patel    | 89         | 90              |
|   | Siddharth Jain | 90         | 91              |
|   | Rohan Gupta    | 91         | 92              |
|   | Aarav Mehta    | 92         | 94              |
|   | Priya Desai    | 94         | 96              |
|   | Kabir Khan     | 96         | 97              |
|   | Ishita Joshi   | 97         | 98              |
|   | Arjun Verma    | 98         | NULL            |

**Q5. Assign students into 4 performance groups using NTILE().**

**Answer:**

```
SELECT
    student_id,
    name,
    score,
    NTILE(4) OVER (ORDER BY score DESC)
    AS performance_group
FROM StudentPerformance;
```

269 • **SELECT**

270     student\_id,

271     name,

272     score,

273     NTILE(4) OVER (ORDER BY score DESC) AS performance\_group

274     FROM StudentPerformance;

275

276

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

|   | student_id | name           | score | performance_group |
|---|------------|----------------|-------|-------------------|
| ▶ | 107        | Arjun Verma    | 95    | 1                 |
|   | 114        | Ishita Joshi   | 93    | 1                 |
|   | 110        | Priya Desai    | 92    | 1                 |
|   | 103        | Kabir Khan     | 91    | 1                 |
|   | 113        | Rohan Gupta    | 89    | 2                 |
|   | 101        | Aarav Mehta    | 88    | 2                 |
|   | 111        | Siddharth Jain | 85    | 2                 |
|   | 104        | Tanvi Patel    | 84    | 2                 |
|   | 108        | Meera Pillai   | 82    | 3                 |
|   | 109        | Nikhil Rao     | 79    | 3                 |
|   | 102        | Riya Singh     | 76    | 3                 |
|   | 112        | Sneha Kulkarni | 74    | 3                 |
|   | 106        | Dev Sharma     | 73    | 4                 |
|   | 115        | Yuvraj Rao     | 71    | 4                 |
|   | 105        | Ayesha Khan    | 67    | 4                 |

**Q6. For each course, assign a row number based on attendance (highest first).**

**Answer:**

```
SELECT
    student_id,
    name,
    course,
    attendance,
```

```

ROW_NUMBER() OVER (
    PARTITION BY course
    ORDER BY attendance DESC
) AS row_number
FROM StudentPerformance;

```

276 • **SELECT**

277     student\_id,

278     name,

279     course,

280     attendance,

281     ROW\_NUMBER() OVER (

282         PARTITION BY course

283         ORDER BY attendance DESC

Result Grid | Filter Rows: | Export: | Wrap

|   | student_id | name           | course       | attendance | row_num |
|---|------------|----------------|--------------|------------|---------|
| ▶ | 114        | Ishita Joshi   | Data Science | 97         | 1       |
|   | 101        | Aarav Mehta    | Data Science | 92         | 2       |
|   | 102        | Riya Singh     | Data Science | 85         | 3       |
|   | 109        | Nikhil Rao     | Data Science | 82         | 4       |
|   | 103        | Kabir Khan     | Python       | 96         | 1       |
|   | 111        | Siddharth Jain | Python       | 90         | 2       |
|   | 115        | Yuvraj Rao     | Python       | 84         | 3       |
|   | 105        | Ayesha Khan    | Python       | 81         | 4       |
|   | 110        | Priya Desai    | SQL          | 94         | 1       |
|   | 113        | Rohan Gupta    | SQL          | 91         | 2       |
|   | 104        | Tanvi Patel    | SQL          | 89         | 3       |
|   | 106        | Dev Sharma     | SQL          | 78         | 4       |
|   | 107        | Arjun Verma    | Tableau      | 98         | 1       |
|   | 108        | Meera Pillai   | Tableau      | 87         | 2       |
|   | 112        | Sneha Kulkarni | Tableau      | 83         | 3       |

**Q7. Calculate the number of days each student has been enrolled (from join\_date to today). (Assume current date = '2025-01-01')**

**Answer:**

```

SELECT
    student_id,
    name,
    join_date,
    DATEDIFF('2025-01-01', join_date) AS days_enrolled
FROM StudentPerformance;

```

287 • **SELECT**

288     student\_id,

289     name,

290     join\_date,

291     DATEDIFF('2025-01-01', join\_date) AS days\_enrolled

292     FROM StudentPerformance;

293

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

|   | student_id | name           | join_date  | days_enrolled |
|---|------------|----------------|------------|---------------|
| ▶ | 101        | Aarav Mehta    | 2023-06-12 | 569           |
|   | 102        | Riya Singh     | 2023-07-01 | 550           |
|   | 103        | Kabir Khan     | 2023-06-20 | 561           |
|   | 104        | Tanvi Patel    | 2023-05-30 | 582           |
|   | 105        | Ayesha Khan    | 2023-07-10 | 541           |
|   | 106        | Dev Sharma     | 2023-05-28 | 584           |
|   | 107        | Arjun Verma    | 2023-06-15 | 566           |
|   | 108        | Meera Pillai   | 2023-06-18 | 563           |
|   | 109        | Nikhil Rao     | 2023-07-05 | 546           |
|   | 110        | Priya Desai    | 2023-05-27 | 585           |
|   | 111        | Siddharth Jain | 2023-07-02 | 549           |
|   | 112        | Sneha Kulkarni | 2023-06-10 | 571           |
|   | 113        | Rohan Gupta    | 2023-05-25 | 587           |
|   | 114        | Ishita Joshi   | 2023-06-25 | 556           |
|   | 115        | Yuvraj Rao     | 2023-07-12 | 539           |

**Q8. Format join\_date as “Month Year” (e.g., “June 2023”).**

**Answer:**

```

SELECT
    student_id,
    name,
    DATE_FORMAT(join_date, '%M %Y') AS formatted_join_date
FROM StudentPerformance;

```

294 • **SELECT**

295     student\_id,

296     name,

297     DATE\_FORMAT(join\_date, '%M %Y') AS formatted\_join\_date

298     FROM StudentPerformance;

299

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

|   | student_id | name           | formatted_join_date |
|---|------------|----------------|---------------------|
| ▶ | 101        | Aarav Mehta    | June 2023           |
|   | 102        | Riya Singh     | July 2023           |
|   | 103        | Kabir Khan     | June 2023           |
|   | 104        | Tanvi Patel    | May 2023            |
|   | 105        | Ayesha Khan    | July 2023           |
|   | 106        | Dev Sharma     | May 2023            |
|   | 107        | Arjun Verma    | June 2023           |
|   | 108        | Meera Pillai   | June 2023           |
|   | 109        | Nikhil Rao     | July 2023           |
|   | 110        | Priya Desai    | May 2023            |
|   | 111        | Siddharth Jain | July 2023           |
|   | 112        | Sneha Kulkarni | June 2023           |
|   | 113        | Rohan Gupta    | May 2023            |
|   | 114        | Ishita Joshi   | June 2023           |
|   | 115        | Yuvraj Rao     | July 2023           |

**Q9. Replace the city 'Mumbai' with 'MUM' for display purposes.**

**Answer:**

```
SELECT
    student_id,
    name,
    REPLACE(city, 'Mumbai', 'MUM') AS city
FROM StudentPerformance;
```

The screenshot shows a SQL query editor with the following code:

```
300 • SELECT
301     student_id,
302     name,
303     REPLACE(city, 'Mumbai', 'MUM') AS city
304 FROM StudentPerformance;
305
```

Below the editor is the 'Result Grid' showing the output of the query. The columns are student\_id, name, and city. The data is as follows:

| student_id | name           | city      |
|------------|----------------|-----------|
| 101        | Aarav Mehta    | MUM       |
| 102        | Riya Singh     | Delhi     |
| 103        | Kabir Khan     | MUM       |
| 104        | Tanvi Patel    | Bengaluru |
| 105        | Ayesha Khan    | Hyderabad |
| 106        | Dev Sharma     | Pune      |
| 107        | Arjun Verma    | Delhi     |
| 108        | Meera Pillai   | Kochi     |
| 109        | Nikhil Rao     | Chennai   |
| 110        | Priya Desai    | Bengaluru |
| 111        | Siddharth Jain | MUM       |
| 112        | Sneha Kulkarni | Pune      |
| 113        | Rohan Gupta    | Delhi     |
| 114        | Ishita Joshi   | Bengaluru |
| 115        | Yuvraj Rao     | Hyderabad |

**Q10. For each course, find the highest score using FIRST\_VALUE().**

**Answer:**

```
SELECT
    student_id,
    name,
    course,
    score,
    FIRST_VALUE(score) OVER (
        PARTITION BY course
        ORDER BY score DESC
    ) AS highest_score
FROM StudentPerformance;
```

The screenshot shows a SQL query editor with the following code:

```
306 • SELECT
307     student_id,
308     name,
309     course,
310     score,
311     FIRST_VALUE(score) OVER (
312         PARTITION BY course
313         ORDER BY score DESC

```

Below the editor is the 'Result Grid' showing the output of the query. The columns are student\_id, name, course, score, and highest\_score. The data is as follows:

| student_id | name           | course       | score | highest_score |
|------------|----------------|--------------|-------|---------------|
| 114        | Ishita Joshi   | Data Science | 93    | 93            |
| 101        | Aarav Mehta    | Data Science | 88    | 93            |
| 109        | Nikhil Rao     | Data Science | 79    | 93            |
| 102        | Riya Singh     | Data Science | 76    | 93            |
| 103        | Kabir Khan     | Python       | 91    | 91            |
| 111        | Siddharth Jain | Python       | 85    | 91            |
| 115        | Yuvraj Rao     | Python       | 71    | 91            |
| 105        | Ayesha Khan    | Python       | 67    | 91            |
| 110        | Priya Desai    | SQL          | 92    | 92            |
| 113        | Rohan Gupta    | SQL          | 89    | 92            |
| 104        | Tanvi Patel    | SQL          | 84    | 92            |
| 106        | Dev Sharma     | SQL          | 73    | 92            |
| 107        | Arjun Verma    | Tableau      | 95    | 95            |
| 108        | Meera Pillai   | Tableau      | 82    | 95            |
| 112        | Sneha Kulkarni | Tableau      | 74    | 95            |